

# Transformer 논문 리뷰\_윤혜주

## Abstract

기존의 시퀀스 변환(sequence transduction) 모델은 복잡한 순환(Recurrent) 또는 합성곱(Convolutional) 신경망을 기반으로 구성되며, 대부분 인코더-디코더 구조를 사용함. 이러한 모델에서 최고 성능을 내는 방법은 인코더와 디코더를 어텐션(attention) 메커니즘으로 연결하는 것이었음.

논문에서는 기존 방식과 달리, **완전히 어텐션 메커니즘만을 사용한 새로운 네트워크 아키텍처인 "Transformer"를 제안함**. 즉, 순환(recurrence)과 합성곱(convolution)을 완전히 제거하고, **순수 어텐션 기반으로 모델을 구성**.

이러한 접근 방식의 장점:

- ✅ **병렬 연산이 가능** → 기존 RNN 기반 모델보다 훨씬 빠르게 학습 가능
- ✅ **고품질 번역 성능** → 기계 번역 성능이 기존 최첨단 모델을 능가

## 실험 결과

- **WMT 2014 영어-독일어(English-to-German) 번역 성능:**
  - Transformer 모델은 **BLEU 28.4점 달성** (기존 최고 모델보다 **2점 이상 향상**)
- **WMT 2014 영어-프랑스어(English-to-French) 번역 성능:**
  - **BLEU 41.8점 기록** (최고 성능 단일 모델 기록 갱신)
  - **8개의 GPU에서 단 3.5일 동안 학습** → 기존 모델 대비 **훨씬 적은 연산 비용**으로 학습 완료
- **다른 NLP 태스크에도 일반화 가능함을 증명**
  - **영어 구문 분석(English Constituency Parsing)에도 성공적으로 적용**
  - **대규모 및 소규모 데이터셋에서도 우수한 성능 유지**

## Introduction

Sequence Modeling과 transduction 문제에서 RNN, long-term memory, gated RNN이 SOTA를 달성해 옴

**Recurrent model**은 parallelization이 불가능해 longer sequence length에서 치명적임

- 최근 연구에서 factorization trick과 conditional computation을 통해 계산 효율성을 많이 개선
- 특히 conditional computation은 모델 성능도 동시에 개선
- > 그러나 여전히 근본적인 sequential computation의 문제는 남아있음

**Attention mechanism**은 다양한 분야의 sequence modeling과 transduction model에서 주요하게 다뤄짐

- Attention mechanism은 input과 output sequence간 길이를 신경쓰지 않아도 됨
- > 하지만 여전히 recurrent network와 함께 사용되었음

## Transformer

- input과 output간 global dependency를 뽑아내기 위해 recurrence를 사용하지 않고, attention mechanism만을 사용함
- > parallelization이 가능해 적은 시간으로 translation quality에서 SOTA를 달성할 수 있었음

## Background

**sequential computation**을 줄이는 것은 Extended Neural GPU, ByteNet, ConvS2S에서도 다뤄짐

- 이 연구들은 모두 CNN을 basic building block으로 사용함
- input output 거리에서 dependency를 학습하기 어려움
- > Transformer에서는 Multi-Head Attention 으로 상수 시간으로 줄어듦

## Self-attention(=intra-attention)

- reading comprehension, abstractive summarization, textual entailment, learning task, independent sentence representations를 포함한 다양한 task에서 성공적으로 사용됨

## End-to-end memory network

- sequence-aligned recurrence 보다 recurrent attention mechanism에 기반함
- simple-language question answering 과 language modeling task에서 좋은 성능을 보임

Transformer는 온전히 self-attention에만 의존한 최초의 transduction model (sequence-aligned RNN이나 Convolution을 사용하지 않음)

## Model Architecture

### Encoder and Decoder Stacks

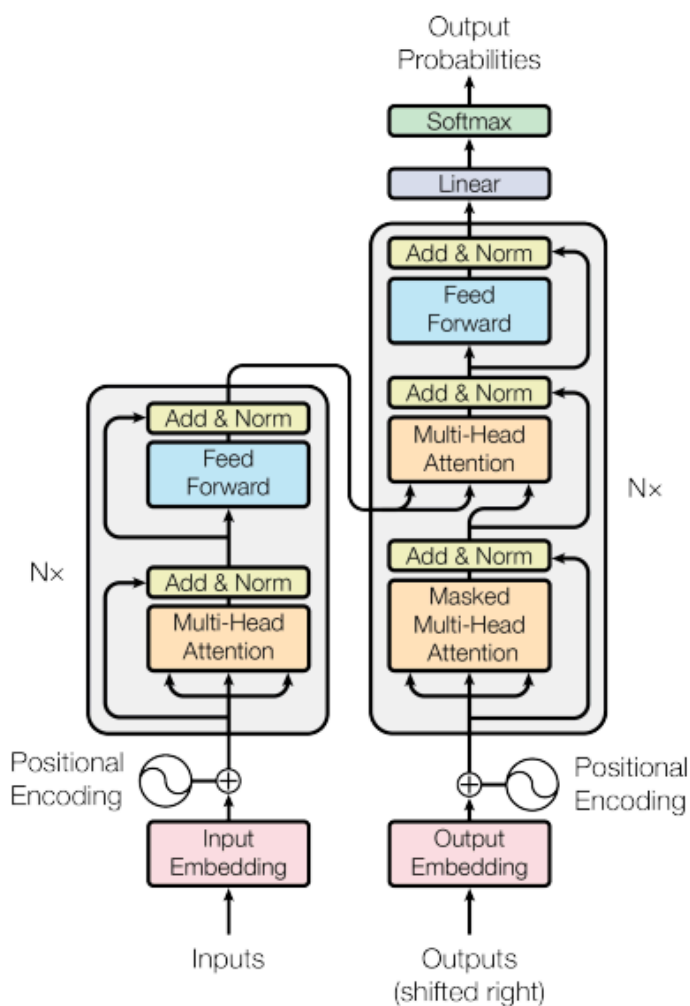


Figure 1: The Transformer - model architecture.

transformer의 구조

### Encoder

- Encoder는 6개의 identical layer로 이루어짐
- 각 layer는 두 개의 sub-layer가 있음

- 첫 번째 sub-layer는 multi-head self-attention mechanism
- 두 번째 sub-layer는 간단한 position-wise fully connected feed-forward network
- 각 two sub-layers 마다 layer normalization 후에 residual connection을 사용함
- 즉 각 sub-layer의 결과는  $LayerNorm(x + Sublayer(x))$  임
- residual connection을 구현하기 위해, embedding layer를 포함한 모든 sub-layer들의 output은 512 차원임
  - $d_{model} = 512$

## Decoder

- Decoder도 마찬가지로 6개의 identical layer로 이루어짐
- 각 Encoder layer의 두 sub-layer에, decoder는 세번째 sub-layer를 추가함
  - encoder stack의 결과에 해당 layer가 multi-head attention을 수행함
- 마찬가지로 residual connection 적용

## Attention

어텐션 메커니즘은 쿼리(query), 키(key), 값(value) 벡터를 입력으로 받아 가중치가 적용된 값 벡터들의 가중 합을 출력

## Scaled Dot-Product Attention

- 쿼리와 키의 내적(dot product)을 계산한 후, 키의 차원 수( $d_k$ )의 제곱근으로 나누어 스케일을 조정
- 소프트맥스를 적용하여 가중치를 구한 후, 이를 값 벡터에 곱하여 최종 출력을 생성한다.
- 일반적인 닷-프로덕트 어텐션과 애디티브 어텐션을 비교했을 때, 닷-프로덕트 어텐션이 더 효율적이며, 특히  $d_k$  값이 클 경우 소프트맥스의 기울기가 작아지는 문제를 해결하기 위해 스케일링을 적용한다.

$d_{kd\_k}$

## Multi-Head Attention

- 입력 벡터를 여러 개의 헤드로 투영하여 병렬적으로 어텐션을 수행한 후, 이를 다시 합쳐 최종 출력을 생성한다.
- 서로 다른 표현 하위 공간에서 정보를 추출할 수 있어 단일 어텐션보다 성능이 향상된다.
- Transformer 모델에서는 8개의 어텐션 헤드를 사용하며, 각 헤드는 64차원의 벡터를 사용하여 전체 연산량이 단일 어텐션과 유사하게 유지된다.

## Applications of Attention in our Model

Transformer는 다음 세 가지 방식으로 다중 헤드 어텐션을 활용한다.

1. 인코더-디코더 attention layers: 디코더가 인코더 출력을 참고하여 어텐션을 수행함으로써 입력 시퀀스의 모든 위치를 고려할 수 있도록
2. 인코더 self-attention layer: 인코더의 각 위치가 이전 레이어의 모든 위치를 참고할 수 있도록 함
3. 디코더 self-attention layer: 디코더가 미래 정보를 참조하지 않도록 마스킹을 적용한 자기-어텐션을 수행

## Position-wise Feed-Forward Networks

- 인코더 및 디코더의 각 레이어에는 두 개의 선형 변환과 ReLU 활성화 함수로 구성된 피드-포워드 신경망이 포함됨
- 동일한 연산을 시퀀스 내 모든 위치에 독립적으로 적용하며, 레이어마다 서로 다른 가중치를 사용

## Embeddings and Softmax

- 입력 및 출력 토큰을 512차원의 벡터로 변환하는 학습된 임베딩을 사용
- 임베딩 레이어와 디코더 출력의 소프트맥스 변환에 동일한 가중치 행렬을 공유하며, 가중치를  $d_{model}$  만큼 곱하여 스케일을 조정

## Positional Encoding

- Transformer는 순차적 구조(예: RNN, CNN)를 사용하지 않으므로, 시퀀스 내 위치 정보를 명시적으로 제공해야 함.
- 이를 위해 사인(sine) 및 코사인(cosine) 함수 기반의 위치 인코딩을 임베딩에 추가.
- 이 방식은 학습된 위치 임베딩과 유사한 성능을 보이면서도, 훈련 중 보지 않은 더 긴 시퀀스에 대해 일반화할 수 있는 장점이 있음.

# Why Self-Attention

## 비교 요소

### 1. 계산 복잡도

- self\_attention:  $O(n^2 \cdot d)$
- 순환 신경망(RNN):  $O(n \cdot d^2)$
- 컨볼루션:  $O(k \cdot n \cdot d^2)$

### 2. 병렬 처리 가능성

- 자기-어텐션은 병렬 연산이 가능하지만, RNN은 순차적으로 실행해야 함.

### 3. 장거리 의존성 학습

- 자기-어텐션은 모든 위치 간 경로 길이를 일정하게 유지하지만, RNN과 CNN은 더 긴 경로를 필요로 함.

## 5. 학습

### 5.1 학습 데이터 및 배치 처리

- WMT 2014 영어-독일어(450만 문장) 및 영어-프랑스어(3600만 문장) 데이터셋을 사용.
- 각 배치는 약 25,000개의 토큰을 포함하는 문장 쌍으로 구성됨.

### 5.2 하드웨어 및 학습 일정

- NVIDIA P100 GPU 8개를 사용하여 학습 진행
- 기본 모델: 12시간(10만 스텝), 대형 모델: 3.5일(30만 스텝)

### 5.3 옵티마이저

- Adam 옵티마이저를 사용하며, 학습률은 훈련 초기에 증가하고 이후 감소하는 방식으로 설정됨.

### 5.4 정규화 기법

- 학습 중 세 가지 정규화 방법을 사용함.

1. 드롭아웃(dropout)
2. 레이어 정규화(layer normalization)
3. 학습률 스케줄링(learning rate scheduling)